

REST

Netværk og distribueret design

Plan

- Hvad er rest?
- Rest api i c#
- Øvelse, Øvelse, Øvelse



REST

- Betyder Representation State Transfer
- REST bruges til at bygge letvægtsservices der er skalerbare og lette at vedligeholde.
- En service, som har REST arkitektur hedder en RESTful service.



REST

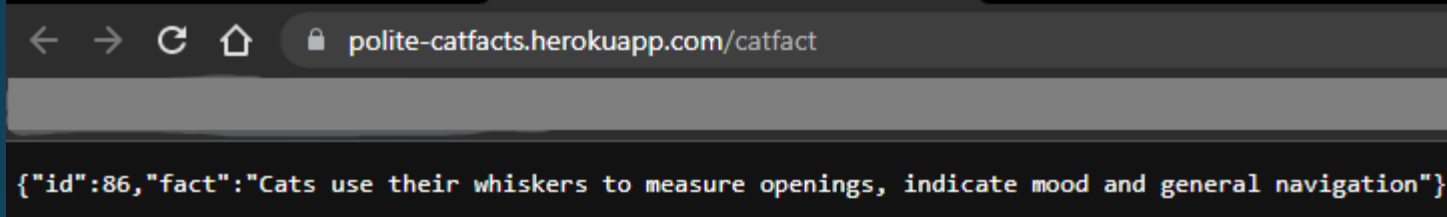
- REST giver adgang til resurser i et bestemt miljø.
 - Kan give adgang til video, billeder eller film som er hostet på en server.
 - Hvis en browser/applikation skal have fat på en af disse resurser, skal der lavet en forespørgsel via http/https til serveren.
 - REST definerer hvordan resurserne kan tilgås
 - For det meste bare et regelsæt for at tilgå databaser.
 - Så vi ikke eksponere klienter direkte til databasen!



**RESTful
Web Services**

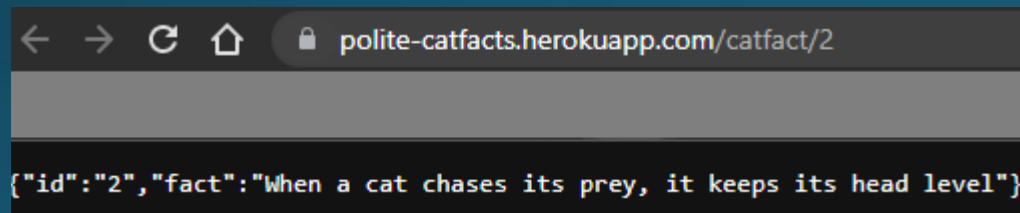
REST - Resources

- En ressource kan f.eks. være brugere i et system, eller andet vilkårligt data
- Hvis vores service f.eks er på <https://catfact.ninja/> og vores site har et rest endpoint på
 - /catfact/{id}
- Giver <https://polite-catfacts.herokuapp.com/catfact> en response (random cat fact)



A screenshot of a web browser window. The address bar shows the URL `polite-catfacts.herokuapp.com/catfact`. The page content displays a JSON response: `{"id":86,"fact":"Cats use their whiskers to measure openings, indicate mood and general navigation"}`.

- Og <https://polite-catfacts.herokuapp.com/catfact/2> giver specifikt catfact med id 2



A screenshot of a web browser window. The address bar shows the URL `polite-catfacts.herokuapp.com/catfact/2`. The page content displays a JSON response: `{"id":"2","fact":"When a cat chases its prey, it keeps its head level"}`.

REST - Request type (Verb)

- Beskriver hvad man vil gøre med resursen.
- En GET request indikerer at man vil have fat i data.
 - Vi ser på andre typer senere
- Som standard i vores browsers laver vi et GET requests
- Dvs. <https://polite-catfacts.herokuapp.com/catfact/2> betyder at browseren kalder en GET for cat fact med id 2

store Access to Petstore orders		
GET	/store/inventory	Returns pet inventories by status
POST	/store/order	Place an order for a pet
GET	/store/order/{orderId}	Find purchase order by ID
DELETE	/store/order/{orderId}	Delete purchase order by ID

REST – Query Parametre

- Parametre kan angives i selve URL'en.
- Serveren læser disse og kan evt. bruge dem til at finde den relevante data

URL-parametre

Request: GET /api/users?id=123&name=John

REST - Request Body

- Data kan også sendes sammen med et request
 - Udover de parametre der er angivet i URL'en.
- Typisk anvendes i et POST-request, men kan også bruges i andre typer af requests. Bruges til at oprette data
- Ved et POST-request beder klienten normalt serveren om at tilføje data til serveren.

```
Request: POST /api/users  
Body: {"id": 123, "name": "John"}
```

REST - Response Body

- Svaret, som serveren giver.
- I vores eksempel med <https://polite-catfacts.herokuapp.com/catfact/2> returneres et katte fakta i json format

```
{"id":"2","fact":"When a cat chases its prey, it keeps its head level"}
```

- I vores browser ser vi blot body, men der er selvfølgelig meget mere i hele svaret ala:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 72
Date: Mon, 5 Jul 2023 12:00:00 GMT

{
  "id": 2,
  "fact": "Cats have five toes on their front paws and four toes on their back paws."
}
```

REST - Response Status codes

- Koder, som returneres til klienten når en respons gives
- Fortæller klienten hvordan requesten gik.
- F.eks. Er kode 200 en kode, som indikerer "alt OK"
 - Fejl kan f.eks. være 404, hvis dataen ikke findes.
- Det er normalt at "reagere" på hvilken type status code man får fra serveren
 - F.eks ved 200 at behandle data og vise til bruger
 - Ved 404 informere om at resursen ikke findes.

REST – http response codes - success

- 200 OK:
 - Indikerer, at anmodningen blev behandlet korrekt, og der er ingen fejl.
- 201 Created:
 - Indikerer, at en ny ressource er blevet oprettet som svar på anmodningen (f.eks. efter en POST-anmodning).
- Dette er blot nogle af dem der findes; fuld oversigt på:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- Det er op til udvikleren at implementere disse så det giver mening.

REST – http response codes - failures

- 400 Bad Request:
 - Indikerer, at anmodningen ikke kunne forstås eller er ugyldig.
- 401 Unauthorized:
 - Indikerer, at anmodningen kræver godkendelse, og at klienten ikke er godkendt (f.eks. logget ind).
- 403 Forbidden
 - Indikerer, at anmodningen er gyldig, men brugeren er ikke tilladt, selvom han er "authorized" (f.eks. bruger ER logget ind, men har ikke konkret tilladelse)
- 404 Not Found
 - Indikerer, at den anmodede ressource ikke blev fundet på serveren.
- 500 Internal Server Error
 - Indikerer, at der opstod en intern serverfejl under behandlingen af anmodningen.

REST – Principper

1. Client-server architecture
 2. Statelessness
 3. Cacheability
 4. Layered system
 5. Uniform interface
- Læs evt. nærmere [her](#)
 - En service kan siges at være RESTful
 - Hvis den opfylder REST-principperne

REST – client/server

- Det mest fundamentale krav til REST arkitekturen.
- Det betyder at en server har en RESTful web service som stiller den korrekte funktionalitet til rådighed.
- Klienten sender anmodninger til serverens RESTful webtjeneste.
 - Anmodningen har et verb (GET, POST, PUT, DELETE), ressourcens URI og data.
- Serveren behandler anmodningen og returnerer et svar.
 - Svar indeholder enten data og/eller en bekræftelse på udført handling.

REST – Stateless

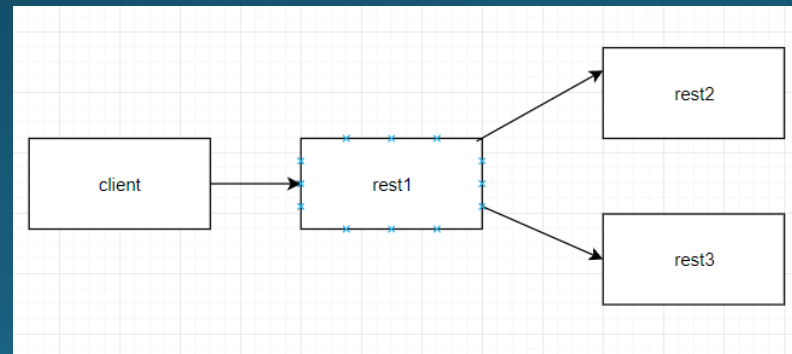
- Betyder at det er op til klienten at sikre at alt den nødvendige information gives til serveren i HVER request.
 - Dvs. hvis man skal bruge authentication, sendes dette med HVER gang
- Serveren gemmer ikke information mellem requests fra en klient.
 - Der gemmes ikke session-data
- Klienten stiller et spørgsmål, serveren besvarer spørgsmålet.

REST – Cacheable

- Nogle gange kan en klient bede serveren om den samme request flere gange.
 - Eller flere klienter beder om det samme
- Requesten går til serveren og serveren sender et svar.
 - Giver netværkstrafik
 - Måske skal server slå op i database?
- Cache er implementeret på klienten eller server
 - Gemmer tidligere svar requests.
 - Dvs. man har et lag, evt. in memory hvor det mest gængse forespørgsler kan ligge, så behøves der ikke et evt. opslag til en database hver gang.
 - Kan også være hvis serveren selv skal hente resursen fra en anden rest service.
 - Sparer netværkstrafik

REST – Layered System

- Klient kan være forbundet direkte til en bestemt server
 - Eller en mellemliggende server, som sender forespørgsel videre
 - f.eks. Være en ekstra service, som klienten interagerer med inden den tager fat i den endelige webservice.
 - F.eks. en cache?
- Klienten kan ikke kende forskel



REST – Uniform Interface

- RESTful API bruger standardiserede HTTP-verber til at håndtere forskellige operationer på ressourcer: POST, GET, PUT, DELETE og PATCH.
- Hver ressource har et unikt URI (Uniform Resource Identifier), der identificerer dens placering på serveren.
 - Eks API/cats eller API/dogs – 2 forskellige typer resurser
- Klienten interagerer med ressourcerne udelukkende gennem URI'erne og de tilhørende HTTP-verber.
 - Ensartet, letforståelig og forudsigelig grænseflade mellem klienten og serveren.

REST – verb metoder - general

- Vi antager at vi har en webservice, som kører på <http://dania.dk/>
 - Vi har derpå et endpoint der hedder /ansatte
 - Så vi tilgår det endpoint via. <http://dania.dk/ansatte>
 - Det endpoint kan tilgås med følgende to verber:
- POST:
 - Vil oprette en ny ansat med de oplysninger, der er indeholdt i body'en i anmodningen.
- GET:
 - Vil returnere en liste over alle ansatte, der er registreret i systemet.
- PUT, PATCH og DELETE vil normalt ikke være implementeret til en generel samling

REST – verb metoder - specifik

- Vores endpoint har også en optionel query-parameter
 - <http://dania.dk/ansatte/{id}>
 - Som kan tilgås med følgende verber:
- GET:
 - vil returnere oplysningerne om den ansatte med det specifikke ID
- PUT:
 - Vil erstatte den ansatte med det specifikke ID med de oplysninger, der er indeholdt i anmodningen.
- PATCH:
 - Vil delvist opdatere den ansatte med det specifikke ID med de oplysninger, der er indeholdt i anmodningen.
- DELETE:
 - Vil slette den ansatte med det specifikke ID fra systemet

REST – hente data

- Når vi bruger vores browser, laver vi et rest kald.
- Mere specifikt laver vores browser et HTTP GET på url'en
 - port 80 ved http
 - Port 443 ved https
- Der er forskellige programmer vi kan bruge til at teste endpoints, f.eks
 - Postman (desktop app) - <https://www.postman.com/>

REST – hente data i .net med HttpClient

- Vi kan selvfølgelig også bruge rest i vores C# apps
- Klassen vi skal have fat i er `HttpClient`
- Vi skal bruge `async/await`, så vi ikke blokerer mens vi venter på svar.
 - Officiel dokumentation på [HttpClient](#)
 - Asynkron programmering i c# [her](#)

```
string url = "http://httpbin.org";
HttpClient client = new HttpClient();
var response = await client.GetAsync(url );

if (response.StatusCode == System.Net.HttpStatusCode.OK)
{
    var responseString = await response.Content.ReadAsStringAsync();
    Console.WriteLine(responseString);
}

Console.WriteLine(response.Content);
```

Microsoft Visual Studio Debug Console

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
  <title>httpbin.org</title>
```

```
  <link href="https://fonts.googleapis.com/css?family=Open+Sans:400,700|Sou
  700"
```

HttpClient - GET

- Vi skal have en URL, altså den adresse vi gerne vil "get" data fra.
- JsonPlaceholder er en testing hjemmeside der returnere Json data og har mange forskellige endpoints.
- I dette tilfælde bruges endpointet til en specifik post med id = 100
- OBS: HttpClient's funktioner kan kaste en `HttpRequestException`

Routes

All HTTP methods are supported. You can use http or https for your requests.

GET	/posts
GET	/posts/1
GET	/posts/1/comments
GET	/comments?postId=1
POST	/posts
PUT	/posts/1
PATCH	/posts/1
DELETE	/posts/1

Endpoints på JsonPlaceholder

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";
```

```
HttpClient httpClient = new HttpClient();
```

```
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);
```

```
if (response.IsSuccessStatusCode)
```

```
{
```

```
    // Læs svaret som en streng
```

```
    string responseContent = await response.Content.ReadAsStringAsync();
```

```
    // Håndter svaret
```

```
    Console.WriteLine($"Svaret fra serveren: {responseContent}");
```

```
}
```

```
else
```

```
{
```

```
    // Håndter fejlresponsen (statuskode er ikke 200)
```

```
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");
```

```
}
```

HttpClient - GET

- Opretter en ny instans af HttpClient-klassen.
- HttpClient er et objekt, der giver mulighed for at foretage HTTP-anmodninger og få adgang til indholdet af de responser, vi modtager.

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";  
HttpClient httpClient = new HttpClient();  
  
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);  
if (response.IsSuccessStatusCode)  
{  
    // Læs svaret som en streng  
    string responseContent = await response.Content.ReadAsStringAsync();  
  
    // Håndter svaret  
    Console.WriteLine($"Svaret fra serveren: {responseContent}");  
}  
else  
{  
    // Håndter fejlresponsen (statuskode er ikke 200)  
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");  
}
```

HttpClient - GET

- Udfører en GET-anmodning til den angivne apiUrl ved hjælp af GetAsync-metoden fra httpClient.
- await betyder, at vi venter på, at anmodningen er fuldført, før vi fortsætter med at køre resten af koden.
- response er en HttpResponseMessage, der repræsenterer svaret fra serveren.

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";  
HttpClient httpClient = new HttpClient();  
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);  
if (response.IsSuccessStatusCode)  
{  
    // Læs svaret som en streng  
    string responseContent = await response.Content.ReadAsStringAsync();  
  
    // Håndter svaret  
    Console.WriteLine($"Svaret fra serveren: {responseContent}");  
}  
else  
{  
    // Håndter fejlresponsen (statuskode er ikke 200)  
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");  
}
```

HttpClient - GET

- Tjekker om anmodningen var successful
 - Er statuskode mellem 200-299?

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";  
HttpClient httpClient = new HttpClient();  
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);  
if (response.IsSuccessStatusCode)  
{  
    // Læs svaret som en streng  
    string responseContent = await response.Content.ReadAsStringAsync();  
  
    // Håndter svaret  
    Console.WriteLine($"Svaret fra serveren: {responseContent}");  
}  
else  
{  
    // Håndter fejlresponsen (statuskode er ikke 200)  
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");  
}
```

HttpClient - GET

- Læser indholdet af svaret som en string.
 - Igen med 'await'

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";

HttpClient httpClient = new HttpClient();

HttpResponseMessage response = await httpClient.GetAsync(apiUrl);
if (response.IsSuccessStatusCode)
{
    // Læs svaret som en string
    string responseContent = await response.Content.ReadAsStringAsync();

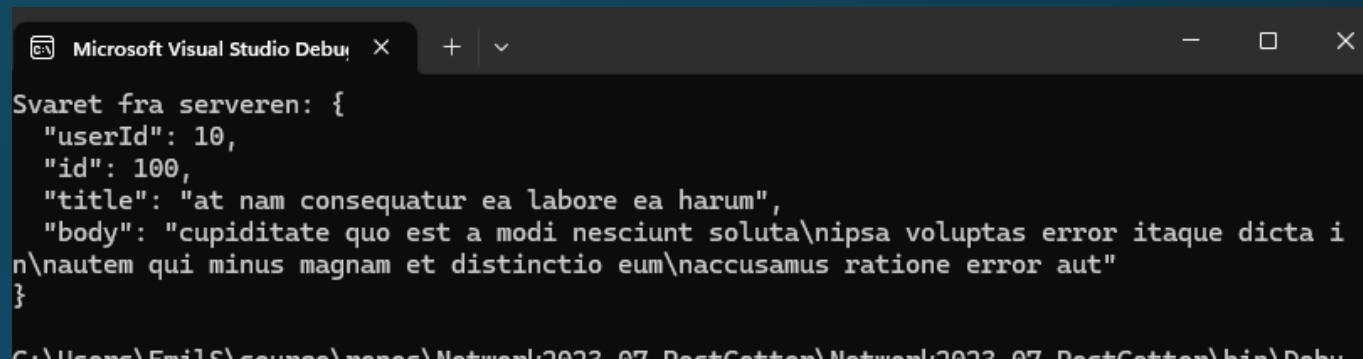
    // Håndter svaret
    Console.WriteLine($"Svaret fra serveren: {responseContent}");
}
else
{
    // Håndter fejlresponsen (statuskode er ikke 200)
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");
}
```

HttpClient – GET kørsel af program

- Vi kan se at Response content er et Json object i formatet:

```
{  
  "userId": int,  
  "id": int,  
  "title": string,  
  "body": string  
}
```

- Dvs vi kender strukturen og kan deserialisere



The screenshot shows a Visual Studio Debug Console window with the title 'Microsoft Visual Studio Debug Console'. It displays a JSON response from a server, labeled 'Svaret fra serveren:'. The JSON object contains fields for 'userId' (10), 'id' (100), 'title' (a long Latin string), and 'body' (another long Latin string).

```
Svaret fra serveren: {  
  "userId": 10,  
  "id": 100,  
  "title": "at nam consequatur ea labore ea harum",  
  "body": "cupiditate quo est a modi nesciunt soluta\nipsa voluptas error itaque dicta i\n\nautem qui minus magnam et distinctio eum\naccusamus ratione error aut"  
}
```

Endpoints på JsonPlaceholder

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";  
  
HttpClient httpClient = new HttpClient();  
  
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);  
if (response.IsSuccessStatusCode)  
{  
    // Læs svaret som en streng  
    string responseContent = await response.Content.ReadAsStringAsync();  
  
    // Håndter svaret  
    Console.WriteLine($"Svaret fra serveren: {responseContent}");  
}  
else  
{  
    // Håndter fejlresponsen (statuskode er ikke 200)  
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");  
}
```

HttpClient – GET autodeserialisering

- Med en meget lille ændring kan vi få dataen ud som en veldefineret c# objekt
 - Kræver at vi definerer klassen/typesen der passer til det forventede svar

```
// Læs svaret som en streng
HttpResponseFromJson responseContent =
    await response.Content.ReadFromJsonAsync<HttpResponseFromJson>();

// Håndter svaret
Console.WriteLine($"Svaret fra serveren: {responseContent}");
```

Property	Type	Value
body	string	"cupiditate quo est a modi nesciunt"
id	int	100
title	string	"at nam consequatur ea labore ea h"
userId	int	10

```
public class HttpResponseFromJson
{
    public int id { get; set; }
    public int userId { get; set; }
    public string title { get; set; }
    public string body { get; set; }
}
```

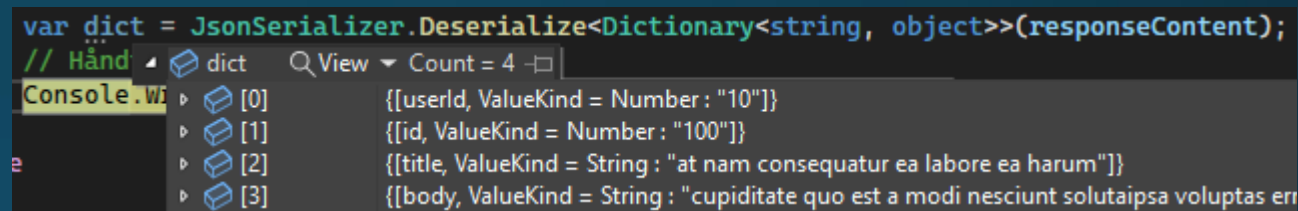
```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";

HttpClient httpClient = new HttpClient();
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);
if (response.IsSuccessStatusCode)
{
    // Læs svaret som en streng
    HttpResponseFromJson responseContent =
        await response.Content.ReadFromJsonAsync<HttpResponseFromJson>();

    // Håndter svaret
    Console.WriteLine($"Svaret fra serveren: {responseContent}");
}
else
{
    // Håndter fejlresponsen (statuskode er ikke 200)
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");
}
```

HttpClient – GET ukendt deserialisering

- Kender vi ikke formatet kan vi altid bare deserialisere som dictionary
 - Eller måske 'dynamic' ?



```
string apiUrl = "https://jsonplaceholder.typicode.com/posts/100";  
  
HttpClient httpClient = new HttpClient();  
HttpResponseMessage response = await httpClient.GetAsync(apiUrl);  
if (response.IsSuccessStatusCode)  
{  
    // Læs svaret som en streng  
    string responseContent = await response.Content.ReadAsStringAsync();  
    var dict = JsonSerializer.Deserialize<Dictionary<string, object>>(responseContent);  
    // Håndter svaret  
    Console.WriteLine($"Svaret fra serveren: {responseContent}");  
}  
else  
{  
    // Håndter fejlresponsen (statuskode er ikke 200)  
    Console.WriteLine($"Anmodningen mislykkedes. Statuskode: {response.StatusCode}");  
}
```

HttpClient – POST

- Angivet URL/endpoint skal tillade 'POST' verbet
- Data der skal postes skal oprettes
- Der kaldes Post i stedet for Get

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts";

HttpClient httpClient = new HttpClient();
HttpJsonPostData postData= new HttpJsonPostData()
{
    userId=1,
    title="Best of Dee Snider",
    body="Some body..."
};
var jsonData = JsonSerializer.Serialize(postData);
StringContent content = new StringContent(jsonData,Encoding.UTF8,"application/json");
HttpResponseMessage response = await httpClient.PostAsync(apiUrl, content);

if (response.IsSuccessStatusCode)
{
    // Håndter succesresponsen
    string responseContent = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"POST-anmodning lykkedes. Respons: {responseContent}");
}
else
{
    // Håndter fejlresponsen (statuskode er ikke 2xx)
    Console.WriteLine($"POST-anmodning mislykkedes. Statuskode: {response.StatusCode}");
}
```

HttpClient – POST

- Angivet URL/endpoint skal tillade 'POST' verbet

Routes

All HTTP methods are supported. You can use http or https for your requests.

GET	/posts
GET	/posts/1
GET	/posts/1/comments
GET	/comments?postId=1
POST	/posts
PUT	/posts/1
PATCH	/posts/1
DELETE	/posts/1

Endpoints på JsonPlaceholder

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts";

HttpClient httpClient = new HttpClient();
HttpJsonPostData postData= new HttpJsonPostData()
{
    userId=1,
    title="Best of Dee Snider",
    body="Some body..."
};
var jsonData = JsonSerializer.Serialize(postData);
StringContent content = new StringContent(jsonData, Encoding.UTF8, "application/json");
HttpResponseMessage response = await httpClient.PostAsync(apiUrl, content);

if (response.IsSuccessStatusCode)
{
    // Håndter succesresponsen
    string responseContent = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"POST-anmodning lykkedes. Respons: {responseContent}");
}
else
{
    // Håndter fejlresponsen (statuskode er ikke 2xx)
    Console.WriteLine($"POST-anmodning mislykkedes. Statuskode: {response.StatusCode}");
}
```

HttpClient – POST

- Oprettelse og serialisering af Json Data.
 - Vigtigt at dette data er i et format endpointet godtager!
 - Eller får vi nok ikke en "god" statuscode.

```
string jsonDataString = @"{
    ""userId"": 1,
    ""title"": ""Best of Dee Snider"",
    ""body"": ""Some body..."
}";
```

Alternativ til at lave object og serialisere.

```
string apiUrl = "https://jsonplaceholder.typicode.com/posts";
HttpClient httpClient = new HttpClient();
HttpJsonPostData postData= new HttpJsonPostData()
{
    userId=1,
    title="Best of Dee Snider",
    body="Some body..."
};
var jsonData = JsonSerializer.Serialize(postData);
StringContent content = new StringContent(jsonData, Encoding.UTF8, "application/json");
HttpResponseMessage response = await httpClient.PostAsync(apiUrl, content);

if (response.IsSuccessStatusCode)
{
    // Håndter succesresponsen
    string responseContent = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"POST-anmodning lykkedes. Respons: {responseContent}");
}
else
{
    // Håndter fejlresponsen (statuskode er ikke 2xx)
    Console.WriteLine($"POST-anmodning mislykkedes. Statuskode: {response.StatusCode}");
}
```

HttpClient – POST

- Oprettelse af vores selve vores content til vores body.
- Vi sender en string, og specificerer den er encoded med utf8 og er i Json formatet.

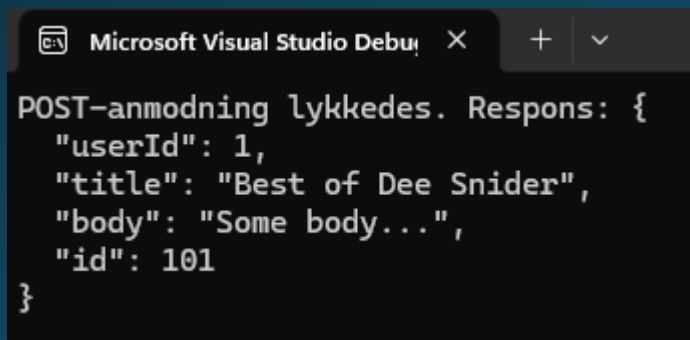
```
string apiUrl = "https://jsonplaceholder.typicode.com/posts";

HttpClient httpClient = new HttpClient();
HttpJsonPostData postData= new HttpJsonPostData()
{
    userId=1,
    title="Best of Dee Snider",
    body="Some body..."
};
var jsonData = JsonSerializer.Serialize(postData);
StringContent content = new StringContent(jsonData, Encoding.UTF8, "application/json");
HttpResponseMessage response = await httpClient.PostAsync(apiUrl, content);

if (response.IsSuccessStatusCode)
{
    // Håndter succesresponsen
    string responseContent = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"POST-anmodning lykkedes. Respons: {responseContent}");
}
else
{
    // Håndter fejlresponsen (statuskode er ikke 2xx)
    Console.WriteLine($"POST-anmodning mislykkedes. Statuskode: {response.StatusCode}");
}
```

HttpClient – POST

- Venter på svar fra serveren, denne gang med content med som data
 - Data bliver sat som body i requesten.
 - Er request gået godt (200-299)
 - Print resultat fra server.



Microsoft Visual Studio Debug Console output:

```
POST-anmodning lykkedes. Respons: {  
  "userId": 1,  
  "title": "Best of Dee Snider",  
  "body": "Some body...",  
  "id": 101  
}
```

Resultat af response fra server.

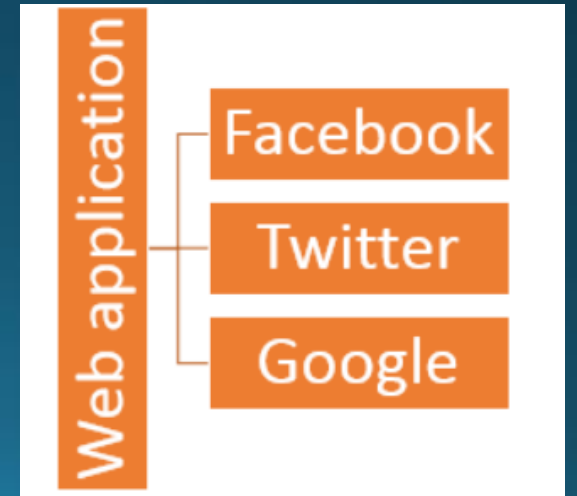
```
string apiUrl = "https://jsonplaceholder.typicode.com/posts";  
  
HttpClient httpClient = new HttpClient();  
HttpJsonPostData postData= new HttpJsonPostData()  
{  
    userId=1,  
    title="Best of Dee Snider",  
    body="Some body..."  
};  
var jsonData = JsonSerializer.Serialize(postData);  
StringContent content = new StringContent(jsonData, Encoding.UTF8, "application/json");  
HttpResponseMessage response = await httpClient.PostAsync(apiUrl, content);  
  
if (response.IsSuccessStatusCode)  
{  
    // Håndter succesresponsen  
    string responseContent = await response.Content.ReadAsStringAsync();  
    Console.WriteLine($"POST-anmodning lykkedes. Respons: {responseContent}");  
}  
else  
{  
    // Håndter fejlresponsen (statuskode er ikke 2xx)  
    Console.WriteLine($"POST-anmodning mislykkedes. Statuskode: {response.StatusCode}");  
}
```

REST – Hvorfor overhovedet?

- Giver applikationer(web, apps, systemer) en standardiseret måde at kommunikere på
 - Dvs. forskellige sprog mulighed for at kommunikere uden uden at kende til hinandens kode.
- Applikationerne kan eksisterer i forskellige miljøer
 - F.eks. Windows og linux
 - Giver det samme resultat uanset miljø

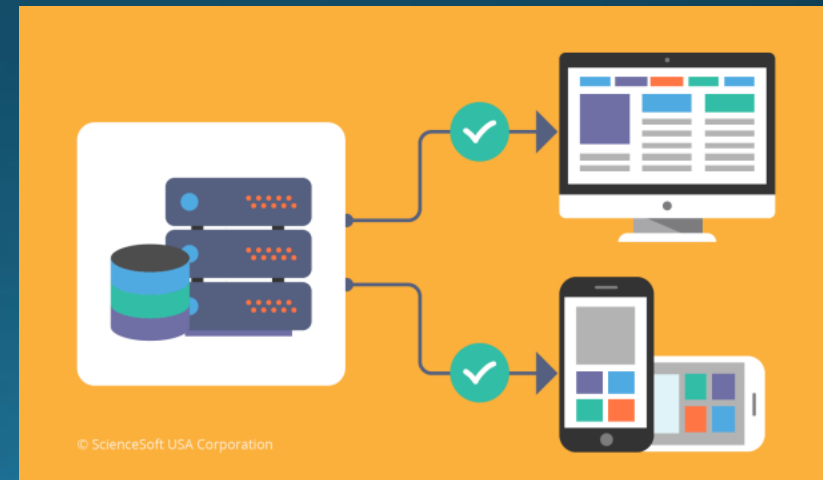
REST – Hvorfor overhovedet?

- Hvis vi laver en applikation, som skal fungere med Facebook, twitter og google
 - Hvilket sprog skal vi så bruge?
 - Skal vi lave interface kode til alle 3 platforme?
- Facebook, twitter og google stiller deres funktionalitet til rådighed gennem Restful webservices.
 - Vi kan lave en klient applikation som kalder disse via REST



REST – Hvorfor overhovedet?

- Mange applikationer skal virke på mobile devices
- At få en applikation til at virke på forskellige devices kræver meget arbejde.
- Restful kan gøre dette meget simpelt
 - Altså det er meget simpelt at dele data uafhængig af at vide hvor, hvordan eller hvem der skal bruge dataen.
- Delt backend, forskellig frontend.



REST – Hvorfor overhovedet?

- Mange applikationer findes som rest services, så det er nemt at interface med dem
 - Spotify, facebook, google, twitter, ChatGPT, Claude AI, etc.
- Nogle er betalte, andre er gratis, det er afhængig af hvad deres formål er.

Øvelser

Øvelse – Requests til API

- Jeg stiller REST service til rådighed
 - 2 API'er til tilgængelige
 - GET /api/characters
 - GET /api/characters/[id]
 - GET /api/wall
 - POST /api/wall
 - Body med 'name' og 'content'
- Hent data fra REST API'erne
 - Kan I deserialisere/parse det til konkrete klasser?
- Post data til REST API
 - Lad brugeren angive navn og besked og lav POST til api

```
{"name": "Alice", "content": "Hello GG Wall"}
```

Øvelse – Udvid API

- Tag udgangspunkt i 'RESTServer' projektet på moodle
 - Udvid 'CharactersController'
 - POST : Tilføjer ny Character
 - Automatisk id?
 - Delete : Fjerner specific Character (evt. ud fra id)
 - Fejlhåndtering hvis id ikke findes?
 - Udvid evt. med andre endpoints
 - Hvordan kan PUT og PATCH bruges?

Øvelse – Rest Authentication! extra

- Implementer authentication til jeres rest service, både role based og login based
- Start med at læse <https://jwt.io/introduction> for information omkring json web token
- Implementer dernæst baseret på følgende code along tutorial(s):
 - <https://www.youtube.com/watch?v=UwruwHI3BIU>
 - <https://www.youtube.com/watch?v=6sMPvucWNRE>